BÀI TẬP SESSION 11: ÔN LUYỆN

BÀI TẬP 1: Library Book Borrowing Management System

Total Marks: 20 | Duration: 90 minutes

Instructions

- Structure your code clearly with proper comments.
- Validate all user inputs (use exceptions or messages).
- Use List<T> to manage book borrowing data.
- Submit all .cs files in a zipped folder named: CSharp_Exam_<YourName>.zip

Question 1: Create an Interface (2 Points)

Task: Create an interface named IBorrowable with the following methods:

Return Type	Method	Description
float	CalculateFee()	Calculates total borrowing fee
void	DisplayInfo()	Displays borrowing information

Question 2: Create the BorrowRecord Class (6 Points)

Task: Create a class BorrowRecord that implements the IBorrowable interface.

Fields and Properties:

Field Name	Type	Description
recordId	int	Unique ID of the borrowing record
borrowerName	string	Name of borrower (5–80 characters)
dailyFee	float	Fee per day (> 0)
daysBorrowed	int	Number of days borrowed (≥ 1)

Requirements:

- A no-argument constructor
- A constructor that accepts all fields and performs validation

Properties:

- RecordId → no validation

- BorrowerName → 5–80 characters
- DailyFee → must be > 0
- DaysBorrowed → must be ≥ 1

Implement Interface Methods:

- CalculateFee() → dailyFee * daysBorrowed (2 Points)
- DisplayInfo() → Example: "Borrower: John | Days: 7 | Total Fee: $35" (1.5 Points)

Validation Rules (2.5 Points):

- Borrower name length 5–80 characters
- Daily fee > 0
- Days borrowed ≥ 1
- Throw ArgumentException or display error messages for invalid input

Question 3: Create PremiumBorrower Class (Inheritance) (4 Points)

Task: Create a subclass PremiumBorrower that extends BorrowRecord.

Additional Field:

- discountRate: float — fixed discount percentage (default: 15%)

Override Methods:

- CalculateFee() → Formula: (dailyFee * daysBorrowed) * (1 - discountRate / 100) (2 Points)
- DisplayInfo() → Example: "Premium Borrower: Anna | Days: 10 | Total Fee: $85 (with discount)" (2 Points)

Question 4: BorrowManager - CRUD Operations (5 Points)

Task: Create a class BorrowManager that manages records using List<BorrowRecord>.

Features:

Function	Description	Points
Add Record / PremiumBorrower	Input from console and validate data	1.5
View All Records	Display all using DisplayInfo()	1.5
Update Daily Fee	Allow updating by recordId	1.0
Delete Record	Remove by recordId with message if not found	1.0

Question 5: Main Class – Console Menu (3 Points)

Task: Create a Main class with a loop-based menu system.

Sample Menu:

```
=== LIBRARY BORROWING MENU ===
1. Add New Borrow Record / Premium Borrower
2. View All Records
3. Update Daily Fee
4. Delete Record
5. Calculate Fee by Record ID
6. Exit
```

- Keep looping until user chooses to exit
- Use try-catch for all user input parsing and invalid options
- Use methods from BorrowManager appropriately (3 Points)

BÀI TẬP 2: Employee Payroll Management System

Total Marks: 20 | Duration: 90 minutes

Instructions

- Structure your code clearly with proper comments.
- Validate all user inputs (use exceptions or messages).
- Use List<T> to manage employee data.
- Submit all .cs files in a zipped folder named: CSharp_Exam_<YourName>.zip

Question 1: Create an Interface (2 Points)

Task: Create an interface named IPayable with the following methods:

Return Type	Method	Description
float	CalculateSalary()	Calculates total salary
void	DisplayInfo()	Displays employee salary information

Question 2: Create the Employee Class (6 Points)

Task: Create a class Employee that implements the IPayable interface.

Fields and Properties:

Field Name	Type	Description
employeeId	int	Unique ID of the employee
fullName	string	Employee full name (5–100 characters)
basicSalary	float	Basic monthly salary (> 0)
workingDays	int	Number of working days (≥ 1)

Requirements:

- A no-argument constructor
- A constructor that accepts all fields and performs validation

Properties:

- EmployeeId → no validation
- FullName → 5–100 characters

- BasicSalary → must be > 0
- WorkingDays → must be ≥ 1

Implement Interface Methods:

- CalculateSalary() → basicSalary * workingDays / 22 (assume 22 working days/month) (2 Points)
- DisplayInfo() → Example: "Employee: John Doe | Days: 20 | Salary: $1800" (1.5 Points)

Validation Rules (2.5 Points):

- Full name length 5–100 characters
- Basic salary > 0
- Working days ≥ 1
- Throw ArgumentException or display error messages for invalid input

Question 3: Create Manager Class (Inheritance) (4 Points)

Task: Create a subclass Manager that extends Employee.

Additional Field:

- allowanceRate: float — allowance percentage (default: 25%)

Override Methods:

- CalculateSalary() → Formula: (basicSalary * workingDays / 22) * (1 + allowanceRate / 100) (2 Points)
- DisplayInfo() → Example: "Manager: Alice | Days: 22 | Salary: $3125 (with allowance)" (2 Points)

Question 4: PayrollManager - CRUD Operations (5 Points)

Task: Create a class PayrollManager that manages employees using List<Employee>.

Features:

Function	Description	Points
Add Employee / Manager	Input from console and validate data	1.5

View All Employees	Display all using DisplayInfo()	1.5
Update Basic Salary	Allow updating by employeeId	1.0
Delete Employee	Remove by employeeId with message if not found	1.0

Question 5: Main Class – Console Menu (3 Points)

Task: Create a Main class with a loop-based menu system.

Sample Menu:

```
=== EMPLOYEE PAYROLL MENU ===
1. Add New Employee / Manager
2. View All Employees
3. Update Basic Salary
4. Delete Employee
5. Calculate Salary by Employee ID
6. Exit
```

- Keep looping until user chooses to exit
- Use try-catch for all user input parsing and invalid options
- Use methods from PayrollManager appropriately (3 Points)